

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Ульяновский государственный технический университет»
Кафедра «Вычислительная техника»

Операционные системы

(название дисциплины)

Лабораторная работа №2

Алгоритмы планирования

(название работы)

Выполнил

Студент группы ИВТИИБД-21

Мокшин Д.И.

(Фамилия И.О.)

Проверил(а):

ст. преподаватель кафедры «ВТ»

(должность)

Беляев К.С.

(Фамилия И.О.)

СОДЕРЖАНИЕ

1. Задание на лабораторную работу.....	3
2. Описание реализации.....	3
3. Тестирование.....	6
На рисунках 3.1-3.6 представлены тесты планировщиков.....	6
Рис 3.1. Список задач.....	6
Рис 3.2. Тест планировщика FCFS.....	7
Рис 3.3. Тест планировщика SJF.....	7
Рис 3.4. Тест планировщика Priority.....	7
Рис 3.5. Тест планировщика Round Robin.....	7
Рис 3.6. Тест планировщика Round Robin + Priority.....	8
4. Вывод.....	8
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	9

1. Задание на лабораторную работу

Необходимо реализовать алгоритмы планирования процессов на языке C с использованием предоставленного фреймворка. Планировщику передаётся заранее определённый набор задач, каждая из которых имеет имя, приоритет и время использования процессора. Необходимо реализовать следующие алгоритмы планирования: FCFS, SJF, приоритетное планирование, циклическое планирование, циклическое планирование с приоритетом.

2. Описание реализации

Загрузка задач из файла осуществляется функцией `load_tasks_from_file()`. Функция открывает файл `schedule.txt`, построчно считывает данные с помощью `fgets()`, удаляет символ новой строки и с помощью `strsep()` разбирает каждую строку по разделителю «,». Первое поле интерпретируется как имя задачи, второе — как приоритет, третье — как время выполнения (`burst`). Для каждой задачи вызывается функция `add_task()`, которая выделяет память, создаёт копию имени и добавляет задачу в связный список с помощью функции `insert()` из модуля `list.h`.

Алгоритм FCFS Рис2.1(First Come First Served) В функции `fcfs_schedule()` задачи выполняются строго в том порядке, в котором они находятся в связном списке (то есть в порядке загрузки из файла). Для каждой задачи вызывается функция `run()` с полным временем выполнения `burst`.

```
void fcfs_schedule(struct node *list) {
    struct node *current = list;
    printf("\nFCFS (First Come First Served)\n");
    while (current != NULL) {
        run(current->task, current->task->burst);
        current = current->next;
    }
    printf("\n\n");
}
```

Рис 2.1 Алгоритм FCFS

Алгоритм SJF Рис 2.2(Shortest Job First) В функции `sjf_schedule()` сначала подсчитывается количество задач, после чего создается динамический массив указателей на структуры `Task`. Задачи копируются в этот массив, сортируются по возрастанию времени выполнения (`burst`) с помощью сортировки пузырьком, а затем выполняются в отсортированном порядке.

```
void sjf_schedule(struct node *list) {
    int count = 0;
    struct node *temp = list;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }

    if (count == 0) return;

    Task **tasks = malloc(count * sizeof(Task*));
    temp = list;
    for (int i = 0; i < count; i++) {
        tasks[i] = temp->task;
        temp = temp->next;
    }

    for (int i = 0; i < count - 1; i++) {
        for (int j = i + 1; j < count; j++) {
            if (tasks[i]->burst > tasks[j]->burst) {
                Task *swap = tasks[i];
                tasks[i] = tasks[j];
                tasks[j] = swap;
            }
        }
    }

    printf("\nSJF (Shortest Job First)\n");
    for (int i = 0; i < count; i++) {
        run(tasks[i], tasks[i]->burst);
    }
    printf("\n\n");
    free(tasks);
}
```

Рис 2.2 Алгоритм SJF

Приоритетное планирование (Priority Scheduling)

Функция `priority_schedule()` работает аналогично SJF, но сортировка задач производится по полю `priority` в порядке убывания приоритета (от более высокого приоритета к более низкому).

Циклическое планирование (Round Robin) В функции `rr_schedule()` используется фиксированный квант времени, равный 10 единицам (константа `QUANTUM`). Алгоритм создает массив задач и массив оставшегося времени `remaining[]`. В цикле задачи поочередно получают процессор на один квант. Если оставшееся время задачи меньше кванта, она выполняется до

завершения. Цикл продолжается до тех пор, пока все задачи не будут выполнены.

```
// priority
void priority_schedule(struct node *list) {
    int count = 0;
    struct node *temp = list;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }

    if (count == 0) return;

    Task **tasks = malloc(count * sizeof(Task*));
    temp = list;
    for (int i = 0; i < count; i++) {
        tasks[i] = temp->task;
        temp = temp->next;
    }

    // Сортировка по приоритету (от большего к меньшему)
    for (int i = 0; i < count - 1; i++) {
        for (int j = i + 1; j < count; j++) {
            if (tasks[i]->priority < tasks[j]->priority) {
                Task *swap = tasks[i];
                tasks[i] = tasks[j];
                tasks[j] = swap;
            }
        }
    }

    printf("\nPriority Scheduling\n");
    for (int i = 0; i < count; i++) {
        run(tasks[i], tasks[i]->burst);
    }
    printf("Завершение Priority\n\n");

    free(tasks);
}
```

Рис 2.3 Приоритетное планирование

Приоритетное планирование с циклическим распределением (Priority + Round Robin) Функция `priority_rr_schedule()` сначала сортирует задачи по приоритету (от высшего к низшему). Затем для каждой группы задач с одинаковым приоритетом (начиная с самого высокого) применяется циклическое планирование с квантом 10. Задачи более высокого приоритета выполняются до полного завершения всех задач данного уровня приоритета, после чего планировщик переходит к следующему (более низкому) уровню приоритета.

3. Тестирование

На рисунках 3.1-3.6 представлены тесты планировщиков.

```
ЗАДАЧИ
T8, 10, 25
T7, 3, 30
T6, 1, 10
T5, 5, 20
T4, 5, 15
T3, 3, 25
T2, 3, 25
T1, 4, 20
```

Рис 3.1. Список задач

```
FCFS (First Come First Served)
Running task = [T8] [10] [25] for 25 units.
Running task = [T7] [3] [30] for 30 units.
Running task = [T6] [1] [10] for 10 units.
Running task = [T5] [5] [20] for 20 units.
Running task = [T4] [5] [15] for 15 units.
Running task = [T3] [3] [25] for 25 units.
Running task = [T2] [3] [25] for 25 units.
Running task = [T1] [4] [20] for 20 units.
```

Рис 3.2. Тест планировщика FCFS

```
SJF (Shortest Job First)
Running task = [T6] [1] [10] for 10 units.
Running task = [T4] [5] [15] for 15 units.
Running task = [T5] [5] [20] for 20 units.
Running task = [T1] [4] [20] for 20 units.
Running task = [T3] [3] [25] for 25 units.
Running task = [T2] [3] [25] for 25 units.
Running task = [T8] [10] [25] for 25 units.
Running task = [T7] [3] [30] for 30 units.
```

Рис 3.3. Тест планировщика SJF

```
Priority Scheduling
Running task = [T8] [10] [25] for 25 units.
Running task = [T5] [5] [20] for 20 units.
Running task = [T4] [5] [15] for 15 units.
Running task = [T1] [4] [20] for 20 units.
Running task = [T3] [3] [25] for 25 units.
Running task = [T2] [3] [25] for 25 units.
Running task = [T7] [3] [30] for 30 units.
Running task = [T6] [1] [10] for 10 units.
Завершение Priority
```

Рис 3.4. Тест планировщика Priority

```
Round Robin (квант = 10)
Running task = [T8] [10] [25] for 10 units.
Running task = [T7] [3] [30] for 10 units.
Running task = [T6] [1] [10] for 10 units.
Running task = [T5] [5] [20] for 10 units.
Running task = [T4] [5] [15] for 10 units.
Running task = [T3] [3] [25] for 10 units.
Running task = [T2] [3] [25] for 10 units.
Running task = [T1] [4] [20] for 10 units.
Running task = [T8] [10] [25] for 10 units.
Running task = [T7] [3] [30] for 10 units.
Running task = [T5] [5] [20] for 10 units.
Running task = [T4] [5] [15] for 5 units.
Running task = [T3] [3] [25] for 10 units.
Running task = [T2] [3] [25] for 10 units.
Running task = [T1] [4] [20] for 10 units.
Running task = [T8] [10] [25] for 5 units.
Running task = [T7] [3] [30] for 10 units.
Running task = [T3] [3] [25] for 5 units.
Running task = [T2] [3] [25] for 5 units.
```

Рис 3.5. Тест планировщика Round Robin

```
Priority + RR (квант = 10)
Running task = [T8] [10] [25] for 10 units
Running task = [T8] [10] [25] for 10 units
Running task = [T8] [10] [25] for 5 units.
Running task = [T5] [5] [20] for 10 units.
Running task = [T5] [5] [20] for 10 units.
Running task = [T4] [5] [15] for 10 units.
Running task = [T4] [5] [15] for 5 units.
Running task = [T1] [4] [20] for 10 units.
Running task = [T1] [4] [20] for 10 units.
Running task = [T3] [3] [25] for 10 units.
Running task = [T3] [3] [25] for 10 units.
Running task = [T3] [3] [25] for 5 units.
Running task = [T2] [3] [25] for 10 units.
Running task = [T2] [3] [25] for 10 units.
Running task = [T2] [3] [25] for 5 units.
Running task = [T7] [3] [30] for 10 units.
Running task = [T7] [3] [30] for 10 units.
Running task = [T7] [3] [30] for 10 units.
Running task = [T6] [1] [10] for 10 units.
```

Рис 3.6. Тест планировщика Round Robin + Priority

4. Вывод

В ходе выполнения лабораторной работы на языке С были реализованы пять алгоритмов планирования процессов с использованием предоставленного фреймворка. Были успешно реализованы следующие алгоритмы:

First Come First Served, Shortest Job First, Priority Scheduling Round Robin
Комбинированный алгоритм приоритетного циклического планирования
Priority + Round Robin

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Керниган Б. У. Язык программирования С / Б. У. Керниган, Д. М. Ричи.
- 2-е изд. - М. : Вильямс, 2021. - 304 с.
2. Робачевский А. М. Операционная система UNIX / А. М. Робачевский. -
2-е изд. - СПб. : БХВ-Петербург, 2010. - 656 с.